

Figure 1: System pipeline from input video to tracked output

```

from sort import Sort
import cv2, torch, numpy as np

# Think of this as giving the system a pair of trained eyes –
# YOLOv5 is a fast, accurate object detector.
model = torch.hub.load('ultralytics/yolov5', 'yolov5s',
    pretrained=True)

# This tracker is what gives the system memory – it helps
# remember who's who across frames.
tracker = Sort()

# Open the input video – one frame at a time, like flipping
# through a digital flipbook.
cap = cv2.VideoCapture('video.mp4')

# Go through the video frame by frame
while cap.isOpened():
    ret, frame = cap.read()
    if not ret: break # End of video

    # Pass the current frame through the object detection model
    results = model(frame)
    detections = []

    # Loop through everything YOLO detected in this frame
    for *xyxy, conf, cls in results.xyxy[0]:
        if int(cls) == 0: # We're only interested in people, not
            # cars, bikes, or dogs.
            x1, y1, x2, y2 = map(int, xyxy)
            # Store the bounding box coordinates and confidence for
            # each detected person
            detections.append([x1, y1, x2, y2, float(conf)])

    # Hand over the list of detections to the tracker – it figures
    # out which ID to assign or keep
    tracked_objects = tracker.update(np.array(detections))

    # For each person being tracked, draw their box and label on
    # the frame
    for x1, y1, x2, y2, track_id in tracked_objects:
        cv2.rectangle(frame, (int(x1), int(y1)), (int(x2),

```

```

int(y2)), (0, 255, 0), 2)
    # Show their ID just above the bounding box – so we know
    # who's who as they move
    cv2.putText(frame, f'ID: {int(track_id)}', (int(x1),
int(y1)-10),
                cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 0), 2)

```

This code doesn't just spot pedestrians. It tracks them like a human driver would—focusing, remembering, and anticipating.

To understand what the system sees initially, Figure 2 depicts a raw video frame that hasn't yet been processed or annotated.



Figure 2: Original input frame showing an unprocessed pedestrian scene

Figure 3 shows how the same frame looks after being processed by the YOLOv5 detector and SORT tracker, with unique IDs and bounding boxes assigned.

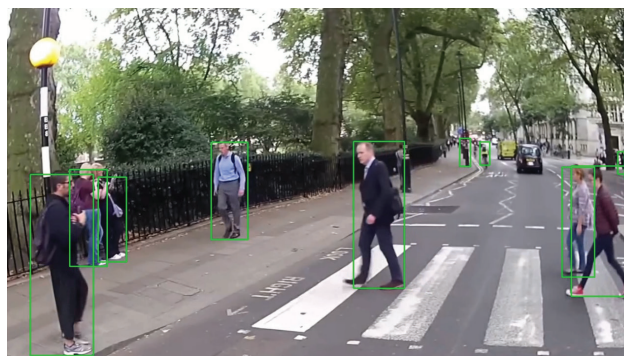


Figure 3: Bounding boxes and ID labels generated during pedestrian tracking

The pedestrian tracking system starts by explaining to the machine how exactly to 'see' its surroundings, which is achieved by YOLOv5, a deep learning model that is trained to recognise various object classes in a particular scene.

The focus is solely on the pedestrian class, and the unpredictable and crucial elements present in road environments. First, the input video is handled in a frame-by-frame sequence; then this model identifies all visible objects